

Experiment 12 : Crisp partitions for the Iris dataset

Objective

Write a Program on implementing crisp partitions for the Iris dataset using a soft computing approach.

Theory

Crisp partitioning (also known as hard clustering) divides a dataset into distinct, non-overlapping groups such that: Each data point belongs to only one cluster Membership is binary (0 or 1) No ambiguity or shared membership exists This is in contrast to fuzzy clustering, where partial membership is allowed.

Program

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from scipy.stats import mode

# Load Iris dataset
iris = load_iris()
X = iris.data
```

```

y_true = iris.target

# Feature scaling (important for distance-based methods)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# K-Means implementation (from scratch for clarity)
class KMeansCrisp:
    def __init__(self, k=3, max_iter=100):
        self.k = k
        self.max_iter = max_iter

    def fit(self, X):
        np.random.seed(42)
        # Initialize centroids randomly
        random_idx = np.random.choice(len(X), self.k, replace=False)
        self.centroids = X[random_idx]

        for _ in range(self.max_iter):
            # Assign clusters (crisp assignment)
            self.labels = self.assign_clusters(X)

            # Compute new centroids
            new_centroids = np.array([
                X[self.labels == i].mean(axis=0) for i in range(self.k)
            ])

            # Check convergence
            if np.allclose(self.centroids, new_centroids):
                break

            self.centroids = new_centroids

    def assign_clusters(self, X):
        distances = np.linalg.norm
        (X[:, np.newaxis] - self.centroids, axis=2)
        return np.argmin(distances, axis=1)

```

```

# Apply K-Means
kmeans = KMeansCrisp(k=3)
kmeans.fit(X_scaled)

# Map clusters to actual labels (for evaluation)
labels = kmeans.labels
new_labels = np.zeros_like(labels)

for i in range(3):
    mask = (labels == i)
    new_labels[mask] = mode(y_true[mask])[0]

# Accuracy
accuracy = accuracy_score(y_true, new_labels)

print("Cluster_Labels:\n", labels)
print("\nAccuracy:", accuracy)

```

Output

Cluster Labels:

```

[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 0 0 0 2 0 0 0 0 0 0 0 0 2 0 0 0 0 2 0 0 0
 0 2 2 2 0 0 0 0 0 0 0 0 2 2 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 2 2 2 0 2 0 2 2
 0 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 0 2 2 2 2 2 0 0 2 2 2 0 2 2 2 0 2 2 2 0 2
 2 0]
Accuracy: 0.8133333333333334

```

Conclusion

About 81 percent of flowers were grouped correctly compared to actual species labels, with clusters numbered 0,1,2